Unidad 1. Servicios de red implicados en el despliegue

Despliegue de Aplicaciones Web (0614) · 2º DAW

IES Hontanilla · Tarancón

Curso 2026-2027

Índice general

1 Servicios de red implicados en el despliegue **1**

Introducción 1

Resultados de aprendizaje y criterios de evaluación 1

Entorno de prácticas 3

Convenciones de este documento 4

1.1 Despliegue de aplicaciones web 4

1.1.1 Qué significa desplegar. Desarrollo, pruebas y producción 4

1.1.2 Arquitecturas web. Modelos, ventajas e inconvenientes 5

1.1.3 Servidores web y servidores de aplicaciones. Clasificación 7

1.1.4 Pilas tecnológicas habituales 8

1.1.5 Estructura y recursos de una aplicación web 9

1.1.6 Requisitos del proceso de implantación 10

1.1.7 Instalación y configuración básica de Apache y Tomcat 11

1.1.8 Pruebas de funcionamiento y documentación del proceso 16

1.2 Sistema de nombres de dominio (DNS) 17

1.2.1 Sistemas de nombres planos y jerárquicos 17

1.2.2 Espacio de nombres: raíz, dominios de primer nivel y siguientes 18

1.2.3 Servidores raíz, autoritativos y recursivos 19

1.2.4 Resolutores. Cómo se resuelve un nombre 20

1.2.5 Registros de recursos 21

1.2.6 Servidor DNS con BIND 9: zonas directa e inversa 22

1.2.7 Parámetros y registros afectados por el despliegue 25

1.2.8 Herramientas de comprobación: dig, nslookup y resolvectl 26

1.3 Servicio de directorio (LDAP) 29

1.3.1 Servicio de directorio: características y funcionalidad 30

1.3.2 Modelo de datos: árbol, entradas, atributos y clases 31

1.3.3 Instalación y archivos básicos de configuración 34

1.3.4 Autenticación de usuarios en el directorio 38

1.3.5 Adaptación del directorio para el despliegue de una aplicación 41

1.3.6 Verificación del funcionamiento y pruebas de rendimiento 44

1.4 Despliegue con contenedores 46
1.4.1 Virtualización de servidores, nube y contenedores 46
1.4.2 Docker: imágenes, contenedores, volúmenes, redes y registros 48
1.4.3 Dockerfile: construir la imagen de una aplicación 50
1.4.4 Docker Compose: aplicaciones de varios contenedores 52
1.4.5 Pruebas de funcionamiento y de rendimiento 55
Cierre de la unidad 56
Resumen 56
Prácticas de la unidad 57
Actividades de repaso 58
Referencias 59

1 Servicios de red implicados en el despliegue

Introducción

Una aplicación web no funciona sola. Para que un usuario la pueda utilizar desde su navegador hace falta un servidor que la ejecute, un nombre de dominio que lleve hasta ese servidor, un mecanismo para saber quién es cada usuario y una forma de instalarla siempre igual, sin errores. Esta unidad estudia esos servicios.

La unidad tiene cuatro apartados:

1. **Despliegue de aplicaciones web.** Qué significa desplegar, cómo se organizan las aplicaciones (arquitecturas), qué servidores intervienen y cómo se instalan Apache y Tomcat.
2. **Sistema de nombres de dominio (DNS).** Cómo se traduce un nombre como `www.daw.test` a una dirección IP y cómo se configura un servidor DNS propio con BIND 9.
3. **Servicio de directorio (LDAP).** Cómo se guardan los usuarios de una organización en un directorio OpenLDAP y cómo lo usan las aplicaciones para comprobar quién es cada usuario.
4. **Despliegue con contenedores.** Cómo empaquetar una aplicación con todo lo que necesita usando Docker y Docker Compose.

Resultados de aprendizaje y criterios de evaluación

La unidad desarrolla dos resultados de aprendizaje (RA) de la programación del módulo.

RA	Descripción	Peso en la nota final
RA1	Implanta arquitecturas web analizando y aplicando criterios de funcionalidad	10 %
RA5	Verifica la ejecución de aplicaciones web comprobando los parámetros de configuración de servicios de red	10 %

Criterio	Contenido	Apartado
RA1.a	Arquitecturas web: características, ventajas e inconvenientes	1.1.2
RA1.b	Fundamentos y protocolos de un servidor web	1.1.3
RA1.c	Instalación y configuración básica de servidores web	1.1.7
RA1.d	Clasificación de los servidores de aplicaciones	1.1.3
RA1.e	Instalación y configuración básica de servidores de aplicaciones	1.1.7
RA1.f	Pruebas de funcionamiento de servidores web y de aplicaciones	1.1.8, 1.4.5
RA1.g	Estructura y recursos de una aplicación web	1.1.5
RA1.h	Requisitos del proceso de implantación	1.1.6
RA1.i	Documentación de la instalación y configuración	1.1.8
RA5.a	Estructura, nomenclatura y funcionalidad de los sistemas de nombres jerárquicos	1.2.1 a 1.2.4
RA5.b	Configuración del servidor de nombres según las aplicaciones desplegadas	1.2.5 a 1.2.8
RA5.c	Función, elementos y estructuras lógicas del servicio de directorio	1.3.1, 1.3.2
RA5.d	Configuración y personalización del servicio de directorio	1.3.3
RA5.e	El directorio como mecanismo de autenticación centralizada	1.3.4

Criterio	Contenido	Apartado
RA5.f	Parámetros del directorio para validar a los usuarios de la aplicación	1.3.5
RA5.g	Documentación de las adaptaciones de los servicios de red	1.2.8, 1.3.6

Entorno de prácticas

Todas las prácticas se hacen en una máquina virtual con Linux, que hace de servidor:

Elemento	Valor
Sistema operativo	Ubuntu Server 24.04 LTS
Nombre del equipo	`srv`
Red	Adaptador 1 en NAT (salida a Internet) y adaptador 2 en red solo-anfitrión
Dirección IP en la red solo-anfitrión	`192.168.56.10`
Dominio de prácticas	`daw.test`

El dominio `.test` está reservado para pruebas (RFC 6761): nunca existirá en Internet, así que no hay riesgo de chocar con un dominio real.

Para fijar la dirección IP del servidor se crea el fichero `/etc/netplan/60-hostonly.yaml`. El nombre de la interfaz (`enp0s8`) se comprueba antes con el comando `ip a`.

```
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [192.168.56.10/24]
```

```
sudo chmod 600 /etc/netplan/60-hostonly.yaml
sudo netplan apply
ip a show enp0s8
```

Convenciones de este documento

- Los comandos que empiezan por `sudo` necesitan permisos de administrador en el servidor Linux.
- El texto en `fuente monoespaciada` es lo que se escribe tal cual: comandos, nombres de fichero, direcciones.
- Cada práctica termina con un apartado **Comprobación**, que indica qué resultado se debe obtener.

1.1 Despliegue de aplicaciones web

1.1.1 Qué significa desplegar. Desarrollo, pruebas y producción

Desplegar una aplicación web es instalarla y configurarla en un servidor para que los usuarios puedan utilizarla. El despliegue incluye:

- preparar el servidor (sistema operativo, red, cortafuegos);
- instalar el software que la aplicación necesita (por ejemplo, Java o PHP y un servidor web);
- copiar la aplicación y configurarla (base de datos, contraseñas, puertos);
- comprobar que funciona y documentar lo que se ha hecho.

Una aplicación pasa normalmente por tres **entornos**, es decir, tres lugares donde se ejecuta con fines distintos:

Entorno	Dónde está	Para qué sirve	Datos
Desarrollo	El equipo del programador	Escribir y probar el código mientras se programa	Inventados
Pruebas (o *staging*)	Un servidor parecido al de producción	Comprobar que la aplicación funciona antes de publicarla	Copia o simulación de los reales
Producción	El servidor definitivo	Dar servicio a los usuarios	Reales

El objetivo es que lo que funciona en desarrollo funcione igual en producción. Muchos fallos de despliegue aparecen porque los entornos son distintos: otra versión de Java, un fichero de configuración que falta, otra ruta de carpetas. Por eso el despliegue se documenta paso a paso y, cuando es posible, se automatiza.

1.1.2 Arquitecturas web. Modelos, ventajas e inconvenientes

La **arquitectura** de una aplicación es la forma en la que se dividen sus partes y en qué equipos se ejecuta cada una. Determina qué hay que instalar y cómo.

Modelo cliente-servidor

Es la base de todas las aplicaciones web:

- El **cliente** (normalmente el navegador) envía una **petición** al servidor.
- El **servidor** es un programa que espera peticiones y devuelve una **respuesta**: una página HTML, una imagen, datos en formato JSON.
- Cliente y servidor se comunican con el protocolo **HTTP**, o con **HTTPS** cuando la comunicación va cifrada.

El servidor espera las peticiones en un **puerto**: un número que identifica a qué programa del equipo va dirigida la petición. Los más habituales son el 80 (HTTP), el 443 (HTTPS) y el 8080, que usa Tomcat por defecto.

- **Ventaja:** los datos y la lógica están en un único sitio. Se actualizan una vez y todos los usuarios ven el cambio.
- **Inconveniente:** si el servidor deja de funcionar, nadie puede usar la aplicación.

Arquitectura en capas (*n-tier*)

La aplicación se divide en capas, cada una con una función:

Capa	Función	Software habitual
Presentación	Lo que ve el usuario: HTML, CSS, imágenes	Servidor web: Apache, Nginx
Lógica de negocio	El programa: cálculos, validaciones, decisiones	Servidor de aplicaciones: Tomcat

Capa	Función	Software habitual
Datos	Almacenamiento permanente de la información	Gestor de bases de datos: MariaDB, PostgreSQL

En una arquitectura de **tres capas** cada una puede estar en un equipo distinto:

```
Navegador --HTTPS--> Apache :443 --> Tomcat :8080 --> MariaDB :3306
(cliente)            (presentación)  (lógica)         (datos)
```

- **Ventajas:** cada capa se configura y se amplía por separado. La base de datos no se expone a Internet: solo la consulta la capa de lógica.
- **Inconvenientes:** hay más componentes que instalar y configurar, y cada salto entre capas añade un pequeño retraso.

Patrón MVC

MVC (Modelo-Vista-Controlador) es una forma de organizar el **código** de una aplicación:

- **Modelo:** las clases que representan los datos y sus reglas.
- **Vista:** las plantillas que generan el HTML.
- **Controlador:** recibe cada petición y decide qué hacer.

MVC no indica en qué servidores se ejecuta cada parte. Una aplicación MVC completa puede desplegarse en un único servidor o repartida en tres capas sin cambiar el código.

Monolito, microservicios y *serverless*

Modelo	Cómo se despliega	Ventajas	Inconvenientes
Monolito	Toda la aplicación en un único paquete, en un servidor	Sencillo de instalar y de probar	Cualquier cambio obliga a desplegarla entera; un fallo afecta a todo

Modelo	Cómo se despliega	Ventajas	Inconvenientes
Microservicios	La aplicación se divide en servicios pequeños e independientes, normalmente cada uno en un contenedor	Cada servicio se actualiza y se amplía por separado; si uno falla, los demás siguen	Muchas más piezas que instalar, comunicar y vigilar
Serverless	El proveedor de la nube ejecuta funciones cuando se necesitan; no hay servidor que administrar	Sin administración de servidores; se paga solo por el uso	Dependencia del proveedor; límites de tiempo de ejecución; retraso en la primera petición

1.1.3 Servidores web y servidores de aplicaciones. Clasificación

Un **servidor web** atiende peticiones HTTP y entrega recursos, sobre todo ficheros estáticos (HTML, CSS, imágenes). También puede reenviar peticiones a otros servidores (proxy inverso) y gestionar el cifrado HTTPS.

Un **servidor de aplicaciones** ejecuta el código de la aplicación: recibe la petición, ejecuta el programa y devuelve el resultado que este genera.

Tipo	Qué hace	Ejemplos
Servidor web	Sirve ficheros estáticos, hace de proxy inverso, cifra con TLS	Apache HTTP Server, Nginx, Microsoft IIS, Caddy
Contenedor de *servlets*	Ejecuta aplicaciones web Java (*servlets*)	Apache Tomcat, Eclipse Jetty
Servidor Jakarta EE completo	Además, ofrece transacciones, mensajería y otros servicios empresariales	WildFly, GlassFish, Payara, Open Liberty, WebLogic
Entorno con servidor integrado	El servidor HTTP va dentro de la propia aplicación	Node.js, Spring Boot (Tomcat embebido), PHP-FPM detrás de un servidor web

En producción se suelen combinar: un servidor web delante, que recibe las peticiones de Internet, y uno o varios servidores de aplicaciones detrás.

1.1.4 Pilas tecnológicas habituales

Una **pila** (*stack*) es el conjunto de programas que, juntos, ejecutan una aplicación. Se nombran con las iniciales de sus componentes:

Pila	Componentes	Uso típico
LAMP	Linux, Apache, MySQL o MariaDB, PHP	WordPress, Moodle, aplicaciones PHP tradicionales
LEMP	Linux, Nginx (se pronuncia *engine-x*), MySQL o MariaDB, PHP	Igual que LAMP, con Nginx como servidor web
Java	Linux, Apache o Nginx, Tomcat, MariaDB o PostgreSQL	Aplicaciones empresariales
Spring Boot	Linux, Java, aplicación Spring Boot con Tomcat embebido, MariaDB o PostgreSQL	Aplicaciones Java actuales, APIs REST, microservicios
MEAN / MERN	MongoDB, Express, Angular o React, Node.js	Aplicaciones en JavaScript de principio a fin

En este módulo se trabaja sobre todo con la pila Java, con Spring Boot y con Apache como servidor web.

Spring Boot

Spring es un *framework* de Java: un conjunto de bibliotecas que resuelven tareas comunes de las aplicaciones, como atender peticiones web, acceder a bases de datos o gestionar la seguridad. **Spring Boot** es la forma de usar Spring con la configuración ya preparada.

Para el despliegue, lo importante de Spring Boot es que **incluye un Tomcat embebido**: el servidor va dentro de la propia aplicación. Eso permite dos formas de desplegarla:

Forma	Qué se entrega	Cómo se arranca	Cuándo se usa
Ejecutable	Un fichero JAR o WAR con Tomcat dentro	`java -jar tienda.war`	Contenedores, microservicios, servidores con una sola aplicación
Desplegada en un Tomcat externo	Un fichero WAR	Se copia en la carpeta `webapps` de Tomcat	Empresas con un Tomcat compartido por varias aplicaciones

Un mismo WAR de Spring Boot sirve para las dos formas. La versión del servidor tiene que ser compatible: Spring Boot 3 y 4 usan los paquetes `jakarta.*` de Jakarta EE, así que necesitan Tomcat 10.1 o superior. Una aplicación antigua, con paquetes `javax.*`, solo funciona en Tomcat 9.

1.1.5 Estructura y recursos de una aplicación web

Una aplicación web está formada por varios tipos de recursos. Cada tipo se trata de forma distinta al desplegar:

Recurso	Qué es	Ejemplos	Quién lo atiende
Estático	Se envía igual a todos los usuarios	HTML, CSS, JavaScript, imágenes	Servidor web
Dinámico	Lo genera el programa en cada petición	Páginas personalizadas, respuestas de una API	Servidor de aplicaciones
Configuración	Ajustes que la aplicación lee al arrancar	`application.properties`, `.env`, `web.xml`	La propia aplicación
Bibliotecas	Código de terceros que usa la aplicación	Ficheros `.jar`, carpeta `vendor/` de PHP	Se incluyen en el paquete o se instalan
Datos	Información que generan los usuarios	Base de datos, ficheros subidos	Servidor de base de datos, almacenamiento

Los datos son lo único que no se puede volver a generar a partir del código. Por eso se hacen copias de seguridad de la base de datos, no de la aplicación.

Aplicaciones Java: el fichero WAR

Una aplicación web Java se entrega al servidor como un fichero **WAR** (*Web Application Archive*). Es un fichero ZIP con una estructura fija:

```
tienda.war
|-- index.html, css/, img/   parte pública: el navegador puede pedirla
|-- META-INF/
|   +-- MANIFEST.MF          información del paquete
+-- WEB-INF/                 parte privada: el servidor nunca la envía
    |-- web.xml              descriptor de despliegue (opcional)
    |-- classes/             código compilado (.class) y configuración
    +-- lib/                 bibliotecas (.jar)
```

Al copiar `tienda.war` en la carpeta `webapps` de Tomcat, Tomcat lo descomprime y publica la aplicación en la dirección `/tienda`. El nombre del fichero determina la dirección; `ROOT.war` se publica en la raíz (`/`).

Como un WAR se puede abrir igual que un ZIP, **no deben guardarse contraseñas dentro de él**. Las contraseñas se configuran en el servidor, por ejemplo con variables de entorno.

Aplicaciones PHP

En una aplicación PHP, el servidor web solo debe tener acceso a la carpeta pública. El resto queda fuera de su alcance:

```
/var/www/tienda/
|-- public/      DocumentRoot de Apache: lo único accesible desde el navegador
|   +-- index.php
|-- src/         código de la aplicación
|-- vendor/      bibliotecas
+-- .env         configuración y contraseñas
```

1.1.6 Requisitos del proceso de implantación

Antes de desplegar una aplicación hay que tener resueltos estos puntos. La lista sirve como plantilla para documentar cualquier despliegue:

1. **Equipo:** procesador, memoria y disco necesarios.
2. **Sistema operativo** y versión.
3. **Entorno de ejecución:** versión de Java, PHP o Node.js. Por ejemplo, Tomcat 11 necesita Java 17 o superior.
4. **Servidor web y servidor de aplicaciones**, con los módulos necesarios.
5. **Base de datos:** gestor, versión, usuario propio de la aplicación y estructura de tablas.
6. **Red:** puertos que se abren y reglas del cortafuegos.
7. **Nombre de dominio** que apunta al servidor (apartado 1.2).
8. **Certificado** para HTTPS (UT2).
9. **Autenticación** de los usuarios (apartado 1.3).
10. **Configuración y contraseñas** fuera del código.
11. **Copias de seguridad, registros (logs) y supervisión.**
12. **Procedimiento de despliegue y de vuelta atrás**, por si la nueva versión falla.

1.1.7 Instalación y configuración básica de Apache y Tomcat

Práctica 1.1. Instalar Apache HTTP Server y Apache Tomcat 11 en el servidor `srv` y comprobar que responden.

Apache HTTP Server

```
sudo apt update
sudo apt install -y apache2
systemctl status apache2 --no-pager
```

Ficheros y carpetas principales de Apache en Ubuntu:

Elemento	Ubicación
Configuración principal	`/etc/apache2/apache2.conf`
Puertos en los que escucha	`/etc/apache2/ports.conf`
Sitios disponibles y activos	`/etc/apache2/sites-available/` y `/etc/apache2/sites-enabled/`
Módulos disponibles y activos	`/etc/apache2/mods-available/` y `/etc/apache2/mods-enabled/`

Elemento	Ubicación
Carpeta de los ficheros del sitio	`/var/www/html`
Registros	`/var/log/apache2/access.log` y `/var/log/apache2/error.log`

Configuración básica: una página propia y el nombre del servidor.

```
echo "<h1>Servidor srv: Apache</h1>" | sudo tee /var/www/html/index.html
echo "ServerName srv.daw.test" | sudo tee /etc/apache2/conf-available/servername.conf
sudo a2enconf servername
sudo apache2ctl configtest
sudo systemctl reload apache2
```

`apache2ctl configtest` revisa la configuración antes de aplicarla. Debe responder `Syntax OK`.

Comprobación:

```
curl -I http://localhost
```

La respuesta empieza por `HTTP/1.1 200 OK` y contiene la cabecera `Server: Apache/2.4...`.

Apache Tomcat 11

Tomcat es un programa Java, así que primero se instala Java 21:

```
sudo apt install -y openjdk-21-jdk-headless
java -version
```

Tomcat se instala desde el fichero oficial. Se crea un usuario del sistema propio para que Tomcat no se ejecute como administrador:

```
TOMCAT_VER=11.0.27
sudo useradd -r -m -U -d /opt/tomcat -s /bin/false tomcat
cd /tmp
wget https://archive.apache.org/dist/tomcat/tomcat-11/v${TOMCAT_VER}/bin/apache-tomcat-
↪  ${TOMCAT_VER}.tar.gz
sudo tar -xzf apache-tomcat-${TOMCAT_VER}.tar.gz -C /opt/tomcat --strip-components=1
sudo chown -R tomcat:tomcat /opt/tomcat
```

Para que Tomcat arranque solo al encender el servidor, se crea un **servicio de systemd** en `/etc/systemd/system/tomcat.service`:

```
[Unit]
Description=Apache Tomcat 11
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now tomcat
systemctl status tomcat --no-pager
```

Carpetas principales de Tomcat:

Carpeta	Contenido
`/opt/tomcat/bin`	Programas de arranque y parada
`/opt/tomcat/conf`	Configuración: `server.xml` (puertos), `tomcat-users.xml` (usuarios)
`/opt/tomcat/webapps`	Aplicaciones desplegadas (ficheros WAR)
`/opt/tomcat/logs`	Registros, entre ellos `catalina.out`

Para desplegar una aplicación de ejemplo basta con copiar un WAR en `webapps`:

```
cd /tmp
wget https://tomcat.apache.org/tomcat-11.0-doc/appdev/sample/sample.war
```

```
sudo cp sample.war /opt/tomcat/webapps/
sudo chown tomcat:tomcat /opt/tomcat/webapps/sample.war
```

Tomcat detecta el fichero nuevo y lo despliega sin reiniciarse.

Comprobación:

```
curl -I http://localhost:8080
curl -s http://localhost:8080/sample/ | head -n 5
sudo grep -i "Deployment of web application archive" /opt/tomcat/logs/catalina.out
```

Las dos primeras órdenes deben devolver respuestas de Tomcat; la tercera, una línea que indica que `sample.war` se ha desplegado. Desde el equipo anfitrión se puede abrir `http://192.168.56.10:8080/sample/` en el navegador.

Una aplicación Spring Boot: ejecutable y desplegada en Tomcat

Práctica 1.2. Crear una aplicación Spring Boot, ejecutarla con su Tomcat embebido y desplegar el mismo WAR en el Tomcat 11 instalado.

El proyecto se genera con el servicio **start.spring.io**, el mismo que usa el asistente de Spring Tools. Las opciones se pasan en la petición:

```
cd ~
curl -s https://start.spring.io/starter.zip \
  -d type=maven-project -d language=java -d packaging=war -d javaVersion=21 \
  -d groupId=es.hontanilla.daw -d artifactId=tienda -d name=tienda \
  -d packageName=es.hontanilla.daw.tienda -d dependencies=web \
  -o tienda.zip
sudo apt install -y unzip
unzip -q tienda.zip -d tienda
cd tienda
```

Opción	Significado
`packaging=war`	Genera un WAR, desplegable en un Tomcat externo
`javaVersion=21`	Versión de Java del proyecto

Opción	Significado
`dependencies=web`	Añade *Spring Web*: atender peticiones HTTP, con Tomcat embebido
`packageName`	Paquete Java donde irá el código

Se añade una clase que responde en la dirección `/saludo`, en `src/main/java/es/hontanilla/daw/tienda/Saludo`

```
package es.hontanilla.daw.tienda;

import java.time.LocalDateTime;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SaludoController {

    @GetMapping("/saludo")
    public String saludo() {
        return "Hola desde el servidor: " + LocalDateTime.now();
    }
}
```

- `@RestController` indica que la clase atiende peticiones web.
- `@GetMapping("/saludo")` asocia el método a la dirección `/saludo`.

Se compila y se empaqueta con **Maven**, la herramienta de construcción de proyectos Java. El proyecto incluye un Maven propio (`mvnw`), así que no hay que instalarlo. La primera vez descarga las bibliotecas y tarda unos minutos:

```
./mvnw -q package
ls -lh target/*.war
```

Forma 1: ejecutable, con Tomcat embebido. El puerto 8080 ya lo usa el Tomcat instalado, así que se arranca en el 8081:

```
java -jar target/tienda-0.0.1-SNAPSHOT.war --server.port=8081
```

En los mensajes de arranque aparece `Tomcat started on port 8081`: es el Tomcat que va dentro de la aplicación. Desde otra terminal:

```
curl http://localhost:8081/saludo
```

Se detiene con `Ctrl+C`.

Forma 2: desplegada en el Tomcat externo. Se copia el WAR con el nombre que tendrá en la dirección:

```
sudo cp target/tienda-0.0.1-SNAPSHOT.war /opt/tomcat/webapps/tienda.war
sudo chown tomcat:tomcat /opt/tomcat/webapps/tienda.war
```

Comprobación:

```
curl http://localhost:8080/tienda/saludo
```

En los dos casos la respuesta es `Hola desde el servidor:` seguida de la fecha y la hora, que cambian en cada petición. En la forma 2 la dirección incluye `/tienda`, que es el nombre del fichero WAR.

1.1.8 Pruebas de funcionamiento y documentación del proceso

Una vez instalado un servidor hay que comprobar que funciona y medir cómo responde.

Qué se comprueba	Comando	Resultado esperado
El servicio está activo	`systemctl status apache2`	`active (running)`
Escucha en el puerto correcto	`sudo ss -tlnp`	Líneas con `:80` (apache2) y `:8080` (java)
Responde a peticiones	`curl -I http://localhost`	`HTTP/1.1 200 OK`
Errores registrados	`sudo tail /var/log/apache2`	Sin errores nuevos
Rendimiento	`ab -n 1000 -c 20 http://localhost/`	Peticiones por segundo y tiempo medio

`ab` (Apache Benchmark) envía muchas peticiones seguidas y mide la respuesta. Se instala con `sudo apt install -y apache2-utils`. En el resultado interesan tres datos:

- `Requests per second`: peticiones atendidas por segundo;
- `Time per request`: tiempo medio de cada petición;
- `Failed requests`: peticiones fallidas, que deben ser 0.

```
ab -n 1000 -c 20 http://localhost/
ab -n 1000 -c 20 http://localhost:8080/sample/
```

Documentación del despliegue

Todo despliegue se documenta para que otra persona pueda repetirlo. Un documento de despliegue tiene, como mínimo, estos apartados:

1. Objetivo y entorno: equipo, sistema operativo, red y direcciones IP.
2. Software instalado, con sus versiones.
3. Pasos realizados, con los comandos exactos.
4. Ficheros de configuración modificados y el motivo de cada cambio.
5. Pruebas realizadas y sus resultados.
6. Problemas encontrados y cómo se resolvieron.

1.2 Sistema de nombres de dominio (DNS)

Los equipos de una red se comunican mediante **direcciones IP** (por ejemplo, `192.168.56.10`). Las personas, en cambio, usan **nombres** (`www.daw.test`). El **DNS** (*Domain Name System*) es el servicio que traduce los nombres en direcciones IP. Sin él, para abrir una web habría que conocer la dirección IP de su servidor.

Para desplegar una aplicación hay que conseguir que su nombre lleve hasta el servidor correcto. Este apartado explica cómo funciona el DNS y cómo se configura un servidor DNS propio.

1.2.1 Sistemas de nombres planos y jerárquicos

	Sistema plano	Sistema jerárquico
Estructura	Una lista única de nombres, sin niveles	Un árbol de niveles separados por puntos

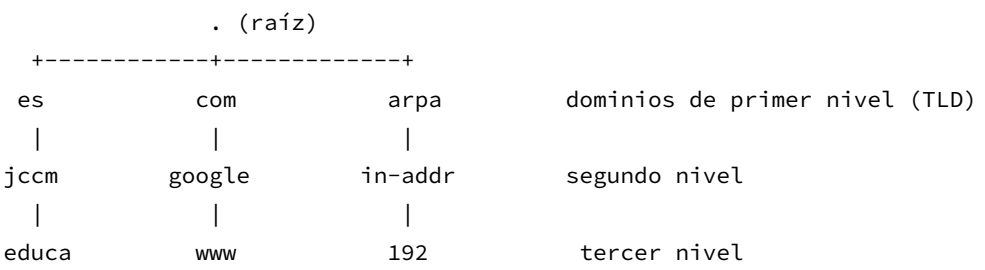
	Sistema plano	Sistema jerárquico
Ejemplos	El fichero `hosts`; los nombres NetBIOS de Windows	DNS
Crecimiento	Difícil: la lista completa debe estar en todos los equipos	Sencillo: cada nivel lo administra una organización distinta
Nombres repetidos	Hay que evitarlos en toda la lista	Solo tienen que ser únicos dentro de su nivel

Antes del DNS, los equipos de Internet usaban un único fichero, `HOSTS.TXT`, que se descargaban todos. Cuando la red creció, esto dejó de ser posible y en 1983 se creó el DNS. Las normas actuales son los RFC 1034 y 1035.

El fichero `hosts` sigue existiendo y se consulta **antes** que el DNS. Está en `/etc/hosts` en Linux y en `C:\Windows\System32\drivers\etc\hosts` en Windows. Es útil para pruebas rápidas en un solo equipo.

1.2.2 Espacio de nombres: raíz, dominios de primer nivel y siguientes

El espacio de nombres del DNS es un árbol:



- **FQDN** (nombre de dominio completo): el nombre entero hasta la raíz, por ejemplo `www.educa.jccm.es.`. El punto final representa la raíz y normalmente no se escribe.
- **Etiqueta:** cada parte entre puntos. Tiene como máximo 63 caracteres, y el nombre completo, 253.
- **Dominio:** una rama del árbol. `jccm.es` incluye todo lo que hay por debajo de él.
- **Zona:** la parte del árbol que administra un servidor concreto. El responsable de un dominio puede **delegar** un subdominio en otro servidor, que pasa a ser una zona aparte.

Tipos de dominios de primer nivel (TLD):

Tipo	Ejemplos	Quién los gestiona
Genéricos (gTLD)	`.com`, `.org`, `.net`, `.dev`, `.app`	Distintas empresas, bajo la supervisión de ICANN
Geográficos (ccTLD)	`.es`, `.fr`, `.pt`	Cada país; en España, Red.es
Infraestructura	`.arpa`	IANA; se usa para la resolución inversa
Reservados (RFC 6761)	`.test`, `.example`, `.invalid`, `.localhost`	No existen en Internet; sirven para pruebas

1.2.3 Servidores raíz, autoritativos y recursivos

Tipo de servidor	Función
Raíz	Conoce qué servidores son responsables de cada dominio de primer nivel. Hay 13 identidades (de `a.root-servers.net` a `m.root-servers.net`), cada una con cientos de copias repartidas por el mundo
Autoritativo	Tiene los datos originales de una zona y responde con autoridad sobre ella. Puede ser primario (donde se editan los datos) o secundario (copia del primario)

Tipo de servidor	Función
Recursivo	Hace todo el trabajo de búsqueda en nombre del cliente y guarda las respuestas en una memoria temporal (caché). Por ejemplo, el DNS del proveedor de Internet, `1.1.1.1` o `8.8.8.8`
Reenviador (*forwarder*)	En lugar de buscar por su cuenta, pasa las consultas a otro servidor recursivo

Un mismo programa, como BIND 9, puede actuar a la vez como autoritativo de unas zonas y como recursivo para los equipos de su red.

1.2.4 Resolutores. Cómo se resuelve un nombre

El **resolutor** es la parte del sistema operativo que hace las consultas DNS por las aplicaciones. En Ubuntu es el servicio `systemd-resolved`.

Pasos para resolver `www.educa.jccm.es` cuando ningún servidor tiene la respuesta en caché:

1. El navegador pide la dirección al resolutor del equipo. El resolutor mira el fichero `hosts` y su caché; si no la encuentra, envía una **consulta recursiva** a su servidor DNS: «dame la respuesta final».
2. El servidor recursivo pregunta a un **servidor raíz**. Este no conoce la respuesta, pero indica qué servidores gestionan `.es`.
3. El recursivo pregunta a un servidor de `.es`, que indica qué servidores gestionan `jccm.es`.
4. El recursivo pregunta al servidor **autoritativo** de `jccm.es`, que devuelve la dirección IP de `www.educa.jccm.es`.
5. El recursivo guarda la respuesta en caché durante el tiempo que indica su **TTL** y se la entrega al resolutor.

Las preguntas de los pasos 2 a 4 son **consultas iterativas**: cada servidor responde con lo que sabe o con una indicación de a quién preguntar. El DNS usa el **puerto 53**: en UDP para la mayoría de consultas y en TCP para respuestas grandes.

La **resolución inversa** obtiene el nombre a partir de una dirección IP. Para `192.168.56.10` se consulta el nombre `10.56.168.192.in-addr.arpa`, con los números en orden inverso.

1.2.5 Registros de recursos

Cada dato de una zona es un **registro de recurso** (RR), con este formato:

```
nombre   TTL    clase  tipo   dato
www      3600   IN     A      192.168.56.10
```

Tipo	Qué contiene	Uso en el despliegue
SOA	Datos de la zona: servidor primario, responsable, número de versión, tiempos	Uno por zona, obligatorio
NS	Servidores autoritativos de la zona	Delegar la zona o un subdominio
A	Dirección IPv4 de un nombre	El nombre principal de cada servidor
AAAA	Dirección IPv6 de un nombre	Lo mismo en IPv6
CNAME	Alias: un nombre que apunta a otro nombre	Varios sitios en el mismo servidor (`www`, `tienda` apuntan a `srv`)
MX	Servidor de correo del dominio, con prioridad	Correo de la aplicación (avisos, recuperación de contraseñas)
TXT	Texto libre	SPF del correo; verificación del dominio ante servicios externos
PTR	Nombre de una dirección IP	Resolución inversa
CAA	Qué autoridades pueden emitir certificados para el dominio	Seguridad de HTTPS

Un CNAME no puede convivir con otros registros del mismo nombre y no puede usarse en el nombre de la propia zona.

El registro SOA

Campo	Significado
Servidor primario	Nombre del servidor donde se editan los datos (`ns1.daw.test.`)
Responsable	Correo del administrador, con un punto en lugar de la `@` (`admin.daw.test.`)
Serial	Número de versión de la zona. **Hay que aumentarlo en cada cambio.** Se suele usar el formato `AAAAMMDDnn`
Refresh	Cada cuánto comprueba el secundario si hay cambios
Retry	Cuánto espera el secundario para reintentar si falla la comprobación
Expire	Cuándo deja de responder el secundario si no contacta con el primario
Minimum	Tiempo que se guarda en caché una respuesta negativa («este nombre no existe»)

1.2.6 Servidor DNS con BIND 9: zonas directa e inversa

Práctica 1.3. Instalar BIND 9 en `srv` como servidor autoritativo de `daw.test` y como servidor recursivo para la red del aula.

Instalación

```
sudo apt update
sudo apt install -y bind9 bind9-utils bind9-dnsutils
systemctl status named --no-pager
```

El servicio se llama `named`. Ficheros de configuración:

Fichero	Contenido
`/etc/bind/named.conf`	Fichero principal: incluye los demás

Fichero	Contenido
`/etc/bind/named.conf.options`	Opciones generales: recursión, reenviadores, interfaces
`/etc/bind/named.conf.local`	Declaración de nuestras zonas
`/etc/bind/zones/`	Ficheros de zona (carpeta que se crea en la práctica)

Opciones generales

Fichero `/etc/bind/named.conf.options`:

```
options {
    directory "/var/cache/bind";

    listen-on { 127.0.0.1; 192.168.56.10; };
    listen-on-v6 { none; };

    // Recursión solo para la red del aula
    recursion yes;
    allow-recursion { 127.0.0.1; 192.168.56.0/24; };
    allow-query { any; };

    // Los nombres que no son de daw.test se consultan a estos servidores
    forwarders { 1.1.1.1; 9.9.9.9; };

    dnssec-validation auto;
};
```

La recursión se limita a la red del aula con `allow-recursion`. Un servidor que hace consultas recursivas para cualquiera puede utilizarse para ataques contra terceros.

Declaración de las zonas

Fichero `/etc/bind/named.conf.local`:

```
zone "daw.test" {
    type primary;
    file "/etc/bind/zones/db.daw.test";
```

```
};

zone "56.168.192.in-addr.arpa" {
    type primary;
    file "/etc/bind/zones/db.192.168.56";
};
```

Ficheros de zona

```
sudo mkdir -p /etc/bind/zones
```

Zona directa, `/etc/bind/zones/db.daw.test`:

```
$TTL 3600
@       IN  SOA ns1.daw.test. admin.daw.test. (
                2026101601  ; serial
                3600        ; refresh
                900         ; retry
                604800      ; expire
                300 )       ; minimum

@       IN  NS      ns1.daw.test.

ns1     IN  A       192.168.56.10
srv     IN  A       192.168.56.10
www     IN  CNAME   srv
```

Zona inversa, `/etc/bind/zones/db.192.168.56`:

```
$TTL 3600
@       IN  SOA ns1.daw.test. admin.daw.test. (
                2026101601 3600 900 604800 300 )

@       IN  NS      ns1.daw.test.

10      IN  PTR     srv.daw.test.
```

En los ficheros de zona:

- `@` representa el nombre de la zona;

- un nombre **sin** punto final se completa con el nombre de la zona (`www` equivale a `www.daw.test.`);
- un nombre **con** punto final es completo.

Olvidar el punto final es el error más frecuente: `srv.daw.test` sin punto se convertiría en `srv.daw.test.daw.test.`.

Comprobación de la configuración y arranque

```
sudo named-checkconf
sudo named-checkzone daw.test /etc/bind/zones/db.daw.test
sudo named-checkzone 56.168.192.in-addr.arpa /etc/bind/zones/db.192.168.56
sudo systemctl restart named
sudo journalctl -u named -n 20 --no-pager
```

Comprobación: `named-checkconf` no muestra nada si la configuración es correcta; cada `named-checkzone` termina con `OK`; y en el registro aparece `zone daw.test/IN: loaded serial 2026101601`.

1.2.7 Parámetros y registros afectados por el despliegue

Al desplegar aplicaciones, la zona del dominio cambia con frecuencia. Estas son las situaciones habituales.

Varios sitios en el mismo servidor. Se crea un CNAME para cada sitio. El DNS solo lleva hasta el servidor. Es el servidor web quien distingue qué sitio mostrar, mediante la cabecera `Host` de la petición HTTP: son los **hosts virtuales** de la UT2.

Subdominios para cada servicio. Es habitual separar la aplicación (`app`), su API (`api`) y el entorno de pruebas (`pruebas`).

Migración a otro servidor. El **TTL** indica cuántos segundos pueden guardar los demás servidores una respuesta en su caché. Antes de cambiar la IP de un servidor:

1. bajar el TTL del registro, por ejemplo a 300 segundos;
2. esperar al menos el TTL antiguo;
3. cambiar la dirección IP;
4. cuando todo funcione, volver a subir el TTL.

Validación de certificados. Para emitir un certificado HTTPS, autoridades como Let's Encrypt pueden pedir que se publique un registro TXT con un código en el nombre `_acme-challenge` del dominio. El registro **CAA** indica qué autoridades están autorizadas a emitir certificados para el dominio.

Práctica 1.4. Ampliar la zona `daw.test` con los nombres que se usarán en el resto del curso.

Fichero `/etc/bind/zones/db.daw.test` completo. Hay que **aumentar el serial**:

```
$TTL 3600
@       IN  SOA ns1.daw.test. admin.daw.test. (
                2026101602  ; serial (aumentado)
                3600 900 604800 300 )

@       IN  NS      ns1.daw.test.
@       IN  MX  10  mail.daw.test.
@       IN  TXT     "v=spf1 mx -all"
@       IN  CAA     0 issue "letsencrypt.org"

ns1       IN  A     192.168.56.10
srv       IN  A     192.168.56.10
srv       IN  AAAA  fd00:56::10
mail      IN  A     192.168.56.10
ldap      IN  A     192.168.56.10

www       IN  CNAME srv
tienda    IN  CNAME srv
app       IN  CNAME srv
api       IN  CNAME srv
pruebas   IN  CNAME srv

temporal  60 IN A   192.168.56.10
```

```
sudo named-checkzone daw.test /etc/bind/zones/db.daw.test
sudo rndc reload daw.test
```

`rndc reload` recarga la zona sin reiniciar el servicio.

1.2.8 Herramientas de comprobación: dig, nslookup y resolvectl

Herramienta	Uso	Disponible en
`dig`	Consultas detalladas: muestra la pregunta, la respuesta, el servidor y el tiempo	Linux (paquete `bind9-dnsutils`)
`nslookup`	Consultas sencillas	Linux y Windows
`resolvectl`	Ver y cambiar el servidor DNS que usa el equipo; consultar a través del resolutor del sistema	Linux con `systemd-resolved`

Práctica 1.5. Comprobar el servidor DNS.

```
S=@192.168.56.10
dig $S srv.daw.test
dig $S tienda.daw.test +short
dig $S -x 192.168.56.10 +short
dig $S MX daw.test +short
dig $S CAA daw.test +short
dig $S www.wikipedia.org +short
dig $S noexiste.daw.test
```

Qué hay que observar en la salida de `dig`:

Dato	Dónde aparece	Qué indica
`status: NOERROR` / `NXDOMAIN`	Cabecera	La consulta ha funcionado / el nombre no existe
`flags: aa`	Cabecera	La respuesta es autoritativa: viene del servidor responsable de la zona
`ANSWER SECTION`	Cuerpo	Los registros encontrados, con su TTL
`Query time`	Final	Lo que ha tardado la respuesta
`SERVER`	Final	Qué servidor ha respondido

Para que el propio servidor use su DNS en la red del aula (el cambio se pierde al reiniciar):

```
sudo resolvectl dns enp0s8 192.168.56.10
sudo resolvectl domain enp0s8 daw.test
resolvectl query www.daw.test
```

Desde el equipo Windows del aula:

```
nslookup www.daw.test 192.168.56.10
```

Para seguir el proceso completo de resolución desde la raíz:

```
dig +trace www.educa.jccm.es
```

Comprobación:

- `srv.daw.test` devuelve `192.168.56.10` con el indicador `aa`;
- `tienda.daw.test` devuelve el alias `srv.daw.test.` y su dirección;
- la consulta inversa devuelve `srv.daw.test.`;
- `www.wikipedia.org` se resuelve a través de los reenviadores;
- `noexiste.daw.test` devuelve `NXDOMAIN`.

Documentación de las adaptaciones (RA5.g)

La documentación del servicio DNS debe incluir:

1. el plan de nombres: qué nombre tiene cada servicio y a qué dirección apunta;
2. los ficheros `named.conf.options`, `named.conf.local` y las dos zonas, comentados;
3. el motivo de cada registro añadido;
4. las pruebas con `dig` y sus resultados.

Errores frecuentes

Síntoma	Causa	Solución
`SERVFAIL` en toda la zona	Error de sintaxis en el fichero de zona	Revisar `named-checkzone` y `journalctl -u named`
Aparece `srv.daw.test.daw.test.`	Falta el punto final en un nombre completo	Añadir el punto final
Los cambios no se ven	No se aumentó el serial, no se recargó o la respuesta antigua sigue en caché	Aumentar el serial y ejecutar `rndc reload`; esperar el TTL
`REFUSED` desde otro equipo	Su dirección no está en `allow-recursion` o `allow-query`	Revisar `named.conf.options`

1.3 Servicio de directorio (LDAP)

Una organización tiene muchos usuarios y muchas aplicaciones: la web, la intranet, el servidor de ficheros, el correo. Si cada aplicación guarda sus propios usuarios, cada persona tiene varias contraseñas y, cuando alguien deja la organización, hay que darle de baja en todas partes. Un **servicio de directorio** resuelve este problema: guarda los usuarios en un único lugar y todas las aplicaciones lo consultan para comprobar quién es cada persona.

1.3.1 Servicio de directorio: características y funcionalidad

Un **directorio** es una base de datos especializada que guarda información sobre personas, grupos, equipos y servicios de una organización. Sus características son:

Característica	Directorio	Base de datos relacional
Organización de los datos	Árbol jerárquico	Tablas relacionadas
Pensado para	Muchas lecturas y búsquedas, pocas escrituras	Lecturas y escrituras frecuentes, transacciones
Estructura de los datos	Clases estándar, comunes a todas las organizaciones	La que defina cada aplicación
Uso típico	Usuarios, contraseñas, grupos, libretas de direcciones	Datos de negocio: pedidos, productos, facturas

LDAP (*Lightweight Directory Access Protocol*) es el protocolo con el que las aplicaciones consultan y modifican un directorio. La versión actual es LDAPv3, definida en los RFC 4510 a 4519. Usa estos puertos:

Puerto	Uso
389	LDAP sin cifrar, o cifrado con StartTLS
636	LDAPS: LDAP cifrado con TLS desde el principio

Programas que ofrecen un servicio de directorio LDAP:

Programa	Notas
OpenLDAP	Libre. Es el que se usa en esta unidad
389 Directory Server	Libre, de Red Hat
Microsoft Active Directory	El directorio de las redes Windows. Admite consultas LDAP

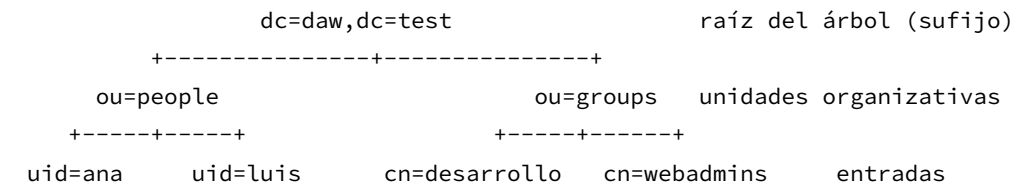
Programa	Notas
FreeIPA	Libre. Combina LDAP con otros servicios de identidad

Autenticación centralizada. Cuando todas las aplicaciones validan a los usuarios contra el mismo directorio:

- cada persona tiene una sola cuenta y una sola contraseña para todo;
- un alta, una baja o un cambio de contraseña se hace una vez y afecta a todas las aplicaciones;
- los grupos del directorio sirven para dar permisos en todas las aplicaciones;
- el directorio pasa a ser un servicio crítico: si falla, nadie puede entrar en ninguna aplicación. Por eso en producción se configuran varias copias (réplicas) y se cifran las comunicaciones.

1.3.2 Modelo de datos: árbol, entradas, atributos y clases

La información del directorio se organiza en un árbol llamado **DIT** (*Directory Information Tree*):



Elemento	Qué es	Ejemplo
Entrada	Cada elemento del árbol	La usuaria Ana
DN (*Distinguished Name*)	Nombre único de una entrada: el camino desde ella hasta la raíz	`uid=ana,ou=people,dc=daw,dc=test`
RDN (*Relative DN*)	La primera parte del DN	`uid=ana`
Atributo	Un dato de la entrada, con tipo y valor. Puede tener varios valores	`mail: ana@daw.test`

Elemento	Qué es	Ejemplo
Clase de objeto (`objectClass`)	Indica qué atributos son obligatorios y cuáles opcionales en una entrada	`inetOrgPerson`
Esquema	Conjunto de clases y atributos que conoce el servidor	`core`, `cosine`, `inetorgperson`, `nis`

Abreviaturas de los atributos más usados:

Atributo	Significado
`dc`	*Domain component*: cada parte del dominio (`dc=daw`,`dc=test` equivale a `daw.test`)
`ou`	*Organizational unit*: unidad organizativa, una carpeta del árbol
`cn`	*Common name*: nombre completo
`sn`	*Surname*: apellido
`uid`	*User id*: nombre de usuario

Clases de objeto que se usan en la unidad:

Clase	Para qué sirve	Atributos obligatorios
`organizationalUnit`	Unidades organizativas (`ou=people`, `ou=groups`)	`ou`
`inetOrgPerson`	Personas: nombre, correo, teléfono	`cn`, `sn`
`posixAccount`	Cuenta de usuario de Linux: número de usuario, carpeta personal	`uid`, `uidNumber`, `gidNumber`, `homeDirectory`

Clase	Para qué sirve	Atributos obligatorios
`posixGroup`	Grupo de Linux, con sus miembros por nombre de usuario (`memberUid`)	`cn`, `gidNumber`
`groupOfNames`	Grupo con sus miembros por DN (`member`). Lo usan Apache, Tomcat y Spring para dar permisos	`cn`, `member`

El formato LDIF

Las entradas y los cambios se escriben en ficheros de texto con formato **LDIF** (*LDAP Data Interchange Format*, RFC 2849). Cada entrada empieza por su DN, sigue con sus atributos y termina con una línea en blanco:

```
dn: uid=ana,ou=people,dc=daw,dc=test
objectClass: inetOrgPerson
uid: ana
cn: Ana García
sn: García
mail: ana@daw.test
```

Operaciones LDAP

Operación	Qué hace	Programa de línea de órdenes
bind	Autenticarse ante el directorio	Todas las órdenes, con `-D` y `-W`
search	Buscar entradas	`ldapsearch`
add	Añadir entradas	`ldapadd`
modify	Modificar atributos	`ldapmodify`
delete	Borrar entradas	`ldapdelete`
Cambio de contraseña	Cambiar la contraseña de una entrada	`ldappasswd`

1.3.3 Instalación y archivos básicos de configuración

Práctica 1.6. Instalar OpenLDAP en `srv` con la base `dc=daw,dc=test` y cargar la estructura, los usuarios y los grupos.

Instalación

```
sudo apt update
sudo apt install -y slapd ldap-utils
sudo dpkg-reconfigure slapd
```

El programa servidor se llama `slapd`. `dpkg-reconfigure` hace estas preguntas:

Pregunta	Respuesta
¿Omitir la configuración del servidor?	No
Nombre de dominio DNS	`daw.test` (la base del árbol será `dc=daw,dc=test`)
Nombre de la organización	`IES Hontanilla DAW`
Contraseña del administrador	Una contraseña de prácticas
¿Borrar la base de datos al desinstalar?	No
¿Mover la base de datos antigua?	Sí

El administrador del directorio es `cn=admin,dc=daw,dc=test`.

Archivos de configuración

Archivo o carpeta	Contenido
`/etc/ldap/slapd.d/`	Configuración del servidor, guardada dentro del propio directorio en la rama `cn=config`. **No se edita a mano:** se modifica con `ldapmodify`

Archivo o carpeta	Contenido
`/etc/ldap/ldap.conf`	Valores por defecto de los programas cliente: servidor y base
`/var/lib/ldap/`	Los datos del directorio
`/etc/ldap/schema/`	Esquemas disponibles

Se configuran los clientes en `/etc/ldap/ldap.conf` para no tener que escribir el servidor y la base en cada orden. El nombre `ldap.daw.test` se añade a la zona DNS del apartado 1.2 (`ldap IN A 192.168.56.10`):

```
BASE    dc=daw,dc=test
URI     ldap://ldap.daw.test
```

Comprobación:

```
systemctl status slapd --no-pager
sudo ss -tlnp | grep 389
ldapsearch -x -LLL -b dc=daw,dc=test
sudo ldapsearch -Q -LLL -Y EXTERNAL -H ldapi:/// -b cn=config dn | grep olcDatabase
```

La tercera orden muestra la entrada raíz `dc=daw,dc=test`. La cuarta muestra la configuración del servidor, a la que solo se accede como administrador del sistema a través de `ldapi:///`, una conexión local.

Carga de datos

Las contraseñas se guardan cifradas con un resumen (*hash*). Se genera el de la contraseña de prácticas:

```
slappasswd -s 'Alumno.2026'
```

La orden devuelve un texto que empieza por `{SSHA}`. Se copia en los ficheros siguientes donde pone `{SSHA}...`.

Fichero `~/ldap/01-estructura.ldif`:

```
dn: ou=people,dc=daw,dc=test
objectClass: organizationalUnit
ou: people

dn: ou=groups,dc=daw,dc=test
objectClass: organizationalUnit
ou: groups
```

Fichero `~/ldap/02-usuarios.ldif`:

```
dn: uid=ana,ou=people,dc=daw,dc=test
objectClass: inetOrgPerson
objectClass: posixAccount
uid: ana
cn: Ana García
sn: García
givenName: Ana
mail: ana@daw.test
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/ana
loginShell: /bin/bash
userPassword: {SSHA}...

dn: uid=luis,ou=people,dc=daw,dc=test
objectClass: inetOrgPerson
objectClass: posixAccount
uid: luis
cn: Luis Martín
sn: Martín
givenName: Luis
mail: luis@daw.test
uidNumber: 10002
gidNumber: 10000
homeDirectory: /home/luis
loginShell: /bin/bash
userPassword: {SSHA}...
```

Fichero `~/ldap/03-grupos.ldif`:

```
dn: cn=desarrollo,ou=groups,dc=daw,dc=test
objectClass: posixGroup
```

```
cn: desarrollo
gidNumber: 10000
memberUid: ana
memberUid: luis

dn: cn=webadmins,ou=groups,dc=daw,dc=test
objectClass: groupOfNames
cn: webadmins
member: uid=ana,ou=people,dc=daw,dc=test
```

```
cd ~/ldap
ldapadd -x -D cn=admin,dc=daw,dc=test -W -f 01-estructura.ldif
ldapadd -x -D cn=admin,dc=daw,dc=test -W -f 02-usuarios.ldif
ldapadd -x -D cn=admin,dc=daw,dc=test -W -f 03-grupos.ldif
```

`-x` indica autenticación simple; `-D` el DN con el que se autentica; `-W` pide la contraseña.

Consultas y modificaciones

```
ldapsearch -x -LLL dn                                          # todas las entradas
ldapsearch -x -LLL "(uid=ana)" cn mail uidNumber               # un usuario
ldapsearch -x -LLL "(&(objectClass=inetOrgPerson)(mail=*@daw.test))" uid
ldapsearch -x -LLL "(|(uid=ana)(uid=luis))" cn
ldapsearch -x -LLL "(cn=webadmins)" member                     # miembros de un grupo
```

Los **filtros** van entre paréntesis: `&` exige que se cumplan todas las condiciones, `|` que se cumpla alguna y `*` es un comodín.

Para modificar una entrada se usa un LDIF con `changetype: modify`. Fichero `04-telefono.ldif`:

```
dn: uid=luis,ou=people,dc=daw,dc=test
changetype: modify
add: telephoneNumber
telephoneNumber: +34 969 000 000
```

```
ldapmodify -x -D cn=admin,dc=daw,dc=test -W -f 04-telefono.ldif
ldappasswd -x -D cn=admin,dc=daw,dc=test -W -S uid=luis,ou=people,dc=daw,dc=test
```

Comprobación: las búsquedas devuelven los dos usuarios y los dos grupos. Una búsqueda sin autenticarse que pida `userPassword` no devuelve la contraseña: lo impiden las reglas de acceso por defecto.

1.3.4 Autenticación de usuarios en el directorio

Para autenticarse ante el directorio se hace una operación **bind**:

Tipo de bind	Cómo funciona
Anónimo	Sin usuario ni contraseña. Solo ve lo que permitan las reglas de acceso
Simple	Con un DN y una contraseña. **La contraseña viaja sin cifrar** si no se usa TLS
SASL	Mecanismos externos: Kerberos, certificados o, en el propio servidor, el usuario del sistema (`EXTERNAL`)

En producción el bind simple siempre se hace sobre **LDAPS** (puerto 636) o **StartTLS** (puerto 389), con un certificado como los de la UT2.

Cómo valida una aplicación a un usuario

El usuario escribe `ana` y su contraseña en un formulario. La aplicación:

1. Se conecta al directorio con una **cuenta de servicio** de solo lectura.
2. **Busca** la entrada con el filtro `(uid=ana)` dentro de `ou=people,dc=daw,dc=test` y obtiene su DN.
3. Hace un **bind con ese DN y la contraseña escrita**. Si funciona, la contraseña es correcta.
4. Si hace falta, comprueba si el DN es miembro de un grupo (`cn=webadmins`) para darle permisos.

La aplicación **nunca lee la contraseña** del directorio: es el servidor LDAP quien la comprueba.

Si la aplicación inserta en el filtro lo que escribe el usuario sin revisarlo, un atacante puede escribir caracteres especiales (`*`, `(`, `)`) para cambiar la búsqueda. Es la **inyección LDAP**, y se evita escapando esos caracteres (RFC 4515) o aceptando solo nombres de usuario con letras y números.

Práctica 1.7. Crear la cuenta de servicio, restringir el acceso al directorio y simular la validación que hace una aplicación.

Cuenta de servicio

```
slappasswd -s 'Lector.2026'
```

Fichero `~/ldap/05-lector.ldif`:

```
dn: cn=lector,dc=daw,dc=test
objectClass: simpleSecurityObject
objectClass: organizationalRole
cn: lector
description: Cuenta de servicio de solo lectura para las aplicaciones
userPassword: {SSHA}...
```

```
ldapadd -x -D cn=admin,dc=daw,dc=test -W -f ~/ldap/05-lector.ldif
```

Reglas de acceso

Por defecto, cualquiera puede leer el directorio (salvo las contraseñas) sin autenticarse. Se cambian las reglas para que solo puedan leer la cuenta de servicio y cada usuario su propia entrada. Fichero `~/ldap/06-acceso.ldif`:

```
dn: olcDatabase={1}mdb,cn=config
changetype: modify
replace: olcAccess
olcAccess: {0}to attrs=userPassword by self write by anonymous auth by * none
olcAccess: {1}to attrs=shadowLastChange by self write by * read
olcAccess: {2}to * by self read by dn.exact="cn=lector,dc=daw,dc=test" read by * none
```

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f ~/ldap/06-acceso.ldif
ldapsearch -x -LLL "(uid=ana)" cn
ldapsearch -x -LLL -D cn=lector,dc=daw,dc=test -W "(uid=ana)" cn
```

- La regla `{0}` permite a cada usuario cambiar su contraseña (`self write`) y a cualquiera **autenticarse** con ella (`anonymous auth`), pero nadie puede leerla.
- La regla `{2}` limita la lectura a la propia entrada y a la cuenta de servicio.
- La primera búsqueda, anónima, ya no devuelve nada; la segunda, con la cuenta de servicio, sí.

Simular la validación de una aplicación

Fichero `~/ldap/valida.sh`:

```
#!/usr/bin/env bash
# Validación de un usuario contra LDAP: búsqueda con la cuenta de servicio + bind con su DN
BASE="ou=people,dc=daw,dc=test"
LECTOR="cn=lector,dc=daw,dc=test"
read -rp "Usuario: " U
read -rsp "Contraseña: " P; echo
read -rsp "Contraseña de la cuenta lector: " LP; echo

# Solo se aceptan letras, números, punto, guion y guion bajo (evita la inyección LDAP)
[[ "$U" =~ ^[a-z0-9._-]+$ ]] || { echo "Usuario no válido"; exit 1; }

DN=$(ldapsearch -x -LLL -D "$LECTOR" -w "$LP" -b "$BASE" "(uid=$U)" dn | sed -n 's/^dn: //p')
[ -z "$DN" ] && { echo "Usuario no encontrado"; exit 1; }

if ldapwhoami -x -D "$DN" -w "$P" > /dev/null 2>&1; then
  echo "Autenticado: $DN"
  ldapsearch -x -LLL -D "$LECTOR" -w "$LP" -b "ou=groups,dc=daw,dc=test" \
    "(&(cn=webadmins)(member=$DN))" dn | grep -q dn && echo "Miembro de webadmins"
else
  echo "Contraseña incorrecta"
fi
```

```
chmod +x ~/ldap/valida.sh
~/ldap/valida.sh
```

Comprobación:

Prueba	Resultado
ana con su contraseña	`Autenticado` y `Miembro de webadmins`
`luis` con su contraseña	`Autenticado`, sin pertenecer a `webadmins`
ana con otra contraseña	`Contraseña incorrecta`
`ana*`	`Usuario no válido`

1.3.5 Adaptación del directorio para el despliegue de una aplicación

Para que una aplicación valide a sus usuarios contra el directorio hay que configurar estos parámetros. Se guardan en la configuración de la aplicación o del servidor, nunca en el código:

Parámetro	Valor en las prácticas
Servidor	`ldap://ldap.daw.test:389` (en producción, `ldaps://…:636`)
Base de los usuarios	`ou=people,dc=daw,dc=test`
Atributo con el nombre de usuario	`uid`
Filtro de búsqueda	`(uid={0})`, donde `{0}` es lo que escribe el usuario
Cuenta de servicio	`cn=lector,dc=daw,dc=test`
Contraseña de la cuenta de servicio	La de prácticas, guardada fuera del código
Base de los grupos	`ou=groups,dc=daw,dc=test`
Atributo de pertenencia al grupo	`member` (DN del usuario)

Así se escriben estos parámetros en los servidores del curso:

Servidor	Configuración	Unidad
Apache	Módulo `mod_authnz_ldap`: `AuthLDAPURL`, `AuthLDAPBindDN`, `AuthLDAPBindPassword`, `Require ldap-group`	UT2
Tomcat	`JNDIRealm` en `server.xml`: `connectionURL`, `userBase`, `userSearch`, `connectionName`, `roleBase`, `roleSearch`	UT3
Spring Boot	Propiedades `spring.ldap.*` y Spring Security	Práctica 1.8

Ejemplo de Apache, que se completará en la UT2:

```
<Location "/intranet">
    AuthType Basic
    AuthName "Intranet DAW"
    AuthBasicProvider ldap
    AuthLDAPURL
↪   "ldap://ldap.daw.test/ou=people,dc=daw,dc=test?uid?sub?(objectClass=inetOrgPerson)"
    AuthLDAPBindDN "cn=lector,dc=daw,dc=test"
    AuthLDAPBindPassword "Lector.2026"
    Require ldap-group cn=webadmins,ou=groups,dc=daw,dc=test
</Location>
```

Práctica 1.8. Validar los usuarios de una aplicación Spring Boot contra el directorio.

Se genera un proyecto con las dependencias *Spring Web*, *Spring Security* y *Spring Data LDAP*:

```
cd ~
curl -s https://start.spring.io/starter.zip \
  -d type=maven-project -d packaging=jar -d javaVersion=21 \
  -d groupId=es.hontanilla.daw -d artifactId=intranet -d name=intranet \
  -d packageName=es.hontanilla.daw.intranet \
  -d dependencies=web,security,data-ldap -o intranet.zip
unzip -q intranet.zip -d intranet && cd intranet
```

Se añade al fichero `pom.xml`, dentro de `<dependencies>`, la biblioteca de Spring Security para LDAP. No hace falta indicar la versión: la fija Spring Boot.

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-ldap</artifactId>
</dependency>
```

Conexión al directorio en `src/main/resources/application.properties`. La contraseña de la cuenta de servicio se lee de una variable de entorno:

```
server.port=8083
spring.ldap.urls=ldap://ldap.daw.test:389
spring.ldap.base=dc=daw,dc=test
spring.ldap.username=cn=lector,dc=daw,dc=test
spring.ldap.password=${LDAP_LECTOR_PASSWORD}
```

Configuración de la seguridad en `src/main/java/es/hontanilla/daw/intranet/SeguridadConfig.java`:

```java
package es.hontanilla.daw.intranet;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.ldap.core.support.BaseLdapPathContextSource;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.ldap.LdapBindAuthenticationManagerFactory;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SeguridadConfig {

    // Todas las direcciones exigen usuario y contraseña (autenticación HTTP Basic)
    @Bean
    SecurityFilterChain filtro(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests(peticiones -> peticiones.anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    // Busca al usuario con (uid={0}) en ou=people y hace bind con su DN y su contraseña
    @Bean
    AuthenticationManager autenticacion(BaseLdapPathContextSource conexion) {
        LdapBindAuthenticationManagerFactory fabrica = new
↪   LdapBindAuthenticationManagerFactory(conexion);
        fabrica.setUserSearchBase("ou=people");
        fabrica.setUserSearchFilter("(uid={0})");
        return fabrica.createAuthenticationManager();
    }
}
```

Controlador `src/main/java/es/hontanilla/daw/intranet/InicioController.java`:

```java
package es.hontanilla.daw.intranet;

import java.security.Principal;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
```

```java
public class InicioController {

    @GetMapping("/")
    public String inicio(Principal usuario) {
        return "Bienvenido a la intranet, " + usuario.getName();
    }
}
```

```
export LDAP_LECTOR_PASSWORD='Lector.2026'
./mvnw -q package
java -jar target/intranet-0.0.1-SNAPSHOT.jar
```

Desde otra terminal:

```
curl -i http://localhost:8083/
curl -i -u ana:Alumno.2026 http://localhost:8083/
curl -i -u ana:otra http://localhost:8083/
```

Comprobación:

Petición	Respuesta
Sin usuario	`HTTP/1.1 401`
`ana` con su contraseña	`HTTP/1.1 200` y `Bienvenido a la intranet, ana`
`ana` con otra contraseña	`HTTP/1.1 401`

La aplicación no tiene ninguna tabla de usuarios: todos están en el directorio. Si se cambia la contraseña de `ana` con `ldappasswd`, la aplicación acepta la nueva sin tocar nada.

1.3.6 Verificación del funcionamiento y pruebas de rendimiento

Qué se comprueba	Orden
El servicio está activo	`systemctl status slapd`

Qué se comprueba	Orden
Escucha en el puerto 389	`sudo ss -tlnp | grep 389`
Responde a búsquedas	`ldapsearch -x -LLL -D cn=lector,dc=daw,dc=test -W "(uid=ana)" dn`
Autentica a un usuario	`ldapwhoami -x -D uid=ana,ou=people,dc=daw,dc=test -W`
Tiempo de 100 autenticaciones	ver abajo
Registro de operaciones	`sudo journalctl -u slapd -f`

Medición del tiempo de 100 autenticaciones seguidas:

```
time (for i in $(seq 100); do
  ldapwhoami -x -D uid=ana,ou=people,dc=daw,dc=test -w 'Alumno.2026' > /dev/null
done)
```

Para ver en el registro cada operación del servidor, se activa el nivel de registro `stats`. Fichero `/tmp/registro.ldif`:

```
dn: cn=config
changetype: modify
replace: olcLogLevel
olcLogLevel: stats
```

```
sudo ldapmodify -Y EXTERNAL -H ldapi:/// -f /tmp/registro.ldif
sudo journalctl -u slapd -f
```

Al repetir `valida.sh` en otra terminal aparecen las operaciones `BIND`, `SRCH` y `RESULT`. `err=0` significa que la operación ha ido bien y `err=49`, credenciales incorrectas.

Documentación de las adaptaciones (RA5.g)

La documentación del servicio de directorio debe incluir:

1. el árbol del directorio dibujado, con sus unidades organizativas;
2. los ficheros LDIF cargados y las reglas de acceso, con su justificación;
3. la cuenta de servicio y los parámetros de validación que usan las aplicaciones (tabla anterior);
4. las pruebas de autenticación realizadas y sus resultados;
5. las mediciones de tiempo.

1.4 Despliegue con contenedores

Un problema frecuente al desplegar es que la aplicación funciona en el equipo del programador y falla en el servidor, porque los dos equipos tienen software distinto. Los **contenedores** resuelven este problema: la aplicación se empaqueta junto con todo lo que necesita y se ejecuta igual en cualquier equipo.

1.4.1 Virtualización de servidores, nube y contenedores

Virtualización

La **virtualización** permite ejecutar varios sistemas operativos aislados en un mismo equipo físico. Cada uno es una **máquina virtual** (VM). El programa que reparte el hardware entre las máquinas virtuales se llama **hipervisor**:

Tipo de hipervisor	Dónde se ejecuta	Ejemplos
Tipo 1 (nativo)	Directamente sobre el hardware, sin otro sistema operativo debajo	VMware ESXi, Microsoft Hyper-V, KVM, Proxmox
Tipo 2 (alojado)	Como un programa más dentro de un sistema operativo	VirtualBox, VMware Workstation

La nube

La **nube** es el alquiler de recursos informáticos a un proveedor, que se pagan según el uso. Según lo que gestiona el proveedor, hay tres modelos (definición del NIST, SP 800-145):

Modelo	El proveedor gestiona	El cliente gestiona	Ejemplos
IaaS (infraestructura como servicio)	Hardware, red y virtualización	Sistema operativo, software, aplicación y datos	AWS EC2, Azure Virtual Machines, Google Compute Engine
PaaS (plataforma como servicio)	Además, el sistema operativo y el entorno de ejecución	Solo la aplicación y sus datos	Azure App Service, Google App Engine, Heroku
SaaS (software como servicio)	Todo	Solo el uso de la aplicación	Gmail, Microsoft 365

Contenedores

Un **contenedor** es un proceso que se ejecuta aislado del resto del sistema, con su propio sistema de ficheros, su propia red y sus propios procesos. A diferencia de una máquina virtual, **comparte el núcleo** (*kernel*) del sistema operativo del equipo. El aislamiento lo proporciona el propio núcleo de Linux con dos mecanismos:

- **espacios de nombres** (*namespaces*): cada contenedor solo ve sus propios procesos, red y ficheros;
- **grupos de control** (*cgroups*): limitan la CPU y la memoria que puede usar cada contenedor.

	Máquina virtual	Contenedor
Qué incluye	Un sistema operativo completo, con su núcleo	Solo la aplicación y sus bibliotecas
Tamaño	Gigabytes	Megabytes
Tiempo de arranque	Minutos	Segundos
Aislamien	Muy fuerte	Menor: comparten el núcleo
Uso principal	Ejecutar sistemas operativos distintos	Desplegar aplicaciones de forma idéntica en cualquier equipo

En la práctica se combinan: los contenedores se ejecutan dentro de máquinas virtuales, en el aula o en la nube.

1.4.2 Docker: imágenes, contenedores, volúmenes, redes y registros

Docker es la herramienta más usada para crear y ejecutar contenedores.

Concepto	Qué es
Imagen	Plantilla de solo lectura con todo lo necesario para ejecutar una aplicación. Está formada por capas
Contenedor	Una imagen en ejecución. Añade una capa propia donde puede escribir
Volumen	Almacenamiento que se conserva aunque se borre el contenedor. Se usa para los datos
Red	Red virtual que comunica contenedores entre sí. Dentro de ella, cada contenedor se localiza por su nombre
Registro	Almacén de imágenes del que se descargan y al que se suben. El principal es Docker Hub
Puerto publicado	Une un puerto del equipo con un puerto del contenedor: `-p 8090:80` lleva el 8090 del equipo al 80 del contenedor

El formato de las imágenes está estandarizado por la **Open Container Initiative** (OCI). Por eso las imágenes de Docker funcionan también con otras herramientas, como Podman o Kubernetes.

Práctica 1.9. Instalar Docker en `srv` y manejar imágenes, contenedores, puertos y volúmenes.

Instalación

Docker se instala desde su repositorio oficial:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
 ↪  /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
 ↪  "$VERSION_CODENAME") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
 ↪  docker-compose-plugin
sudo usermod -aG docker $USER
```

Para que el último comando tenga efecto hay que cerrar la sesión y volver a entrar. Pertenecer al grupo `docker` da, en la práctica, permisos de administrador sobre el equipo: en un servidor real solo se concede a los administradores.

```
docker version
docker run --rm hello-world
```

Órdenes básicas

```
docker pull nginx:stable                          # descargar una imagen
docker images                                     # imágenes descargadas
docker run -d --name web1 -p 8090:80 nginx:stable # crear y arrancar un contenedor
docker ps                                         # contenedores en marcha
curl -I http://localhost:8090                     # probarlo
docker logs web1                                  # mensajes del contenedor
docker exec -it web1 bash                         # abrir una terminal dentro
docker stop web1 && docker rm web1                # pararlo y borrarlo
```

Un contenedor con una página propia, que se lee de una carpeta del equipo (montaje):

```
mkdir -p ~/web1
echo "<h1>Página servida desde un contenedor</h1>" > ~/web1/index.html
docker run -d --name web1 -p 8090:80 -v ~/web1:/usr/share/nginx/html:ro nginx:stable
curl http://localhost:8090
```

Un volumen con nombre, cuyos datos sobreviven al contenedor:

```
docker volume create datos
docker run --rm -v datos:/datos alpine sh -c 'echo "dato guardado" > /datos/f.txt'
docker run --rm -v datos:/datos alpine cat /datos/f.txt
```

Comprobación: `curl` devuelve la página propia y el segundo contenedor muestra `dato guardado`, aunque lo escribió otro contenedor que ya no existe.

1.4.3 Dockerfile: construir la imagen de una aplicación

Un **Dockerfile** es un fichero de texto con las instrucciones para construir una imagen:

Instrucción	Qué hace
`FROM`	Imagen de partida
`WORKDIR`	Carpeta de trabajo dentro de la imagen
`COPY`	Copia ficheros del equipo a la imagen
`RUN`	Ejecuta una orden durante la construcción
`EXPOSE`	Indica en qué puerto escucha la aplicación
`USER`	Usuario con el que se ejecutará la aplicación
`ENTRYPOINT`	Orden que se ejecuta al arrancar el contenedor

Práctica 1.10. Construir la imagen de la aplicación Spring Boot `tienda` de la práctica 1.2, de dos formas.

Forma 1: Spring Boot con su Tomcat embebido

Se usa una **construcción en dos etapas**:

1. La primera etapa tiene el JDK completo y compila el proyecto.
2. La segunda solo tiene lo necesario para ejecutar (JRE) y recibe el WAR ya construido.

La imagen final es más pequeña y no contiene el código fuente ni las herramientas de compilación.

Fichero `~/tienda/Dockerfile`:

```
# Etapa 1: compilar con el JDK
FROM eclipse-temurin:21-jdk AS construccion
WORKDIR /src
COPY . .
RUN ./mvnw -q package -DskipTests

# Etapa 2: ejecutar con el JRE
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=construccion /src/target/tienda-0.0.1-SNAPSHOT.war app.war
RUN groupadd -r app && useradd -r -g app app
USER app
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.war"]
```

Fichero `~/tienda/.dockerignore`, para no copiar a la imagen lo que no hace falta:

```
target
.git
```

```
cd ~/tienda
docker build -t tienda:1.0 .
docker images tienda
docker run -d --name tienda -p 8085:8080 tienda:1.0
docker logs tienda | grep "Tomcat started"
curl http://localhost:8085/saludo
```

Forma 2: el WAR dentro de una imagen de Tomcat

La imagen oficial de Tomcat ya trae Tomcat instalado. Solo hay que copiar el WAR en su carpeta `webapps`. Con el nombre `ROOT.war` se publica en la raíz.

Fichero `~/tienda/Dockerfile.tomcat`:

```
FROM tomcat:11.0-jdk21
COPY target/tienda-0.0.1-SNAPSHOT.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
```

```
./mvnw -q package -DskipTests
docker build -f Dockerfile.tomcat -t tienda-tomcat:1.0 .
docker run -d --name tienda-tomcat -p 8086:8080 tienda-tomcat:1.0
curl http://localhost:8086/saludo
```

Comprobación: las dos órdenes `curl` devuelven `Hola desde el servidor:` y la hora. La aplicación es la misma; cambia dónde está Tomcat: dentro de la aplicación (forma 1) o como servidor que aloja la aplicación (forma 2).

```
docker rm -f tienda tienda-tomcat
```

Publicar la imagen en un registro

Para desplegar la imagen en otro servidor, se sube a un registro y se descarga desde allí. Con una cuenta de Docker Hub:

```
docker login
docker tag tienda:1.0 USUARIO/tienda:1.0
docker push USUARIO/tienda:1.0
```

En el servidor de destino basta con `docker run USUARIO/tienda:1.0`: no hace falta instalar Java ni Maven.

1.4.4 Docker Compose: aplicaciones de varios contenedores

Docker Compose describe en un fichero YAML todos los contenedores de una aplicación, con sus redes, volúmenes y variables, y los arranca con una sola orden:

Orden	Qué hace
`docker compose up -d --build`	Construye las imágenes que falten y arranca todos los servicios
`docker compose ps`	Estado de los servicios
`docker compose logs -f app`	Mensajes de un servicio
`docker compose down`	Para y borra los contenedores y la red; los volúmenes se conservan

Orden	Qué hace
`docker compose down -v`	Además, borra los volúmenes

Compose crea una red para la aplicación en la que cada servicio se localiza **por su nombre**: desde el servicio `web`, la aplicación es `app:8080`. Lo resuelve un DNS interno de Docker.

Práctica 1.11. Desplegar con Compose una arquitectura de tres capas: Nginx como proxy inverso (presentación), la aplicación Spring Boot (lógica) y MariaDB (datos).

Fichero `~/tienda/proxy.conf`, la configuración de Nginx:

```
server {
    listen 80;
    location / {
        proxy_pass http://app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Fichero `~/tienda/.env`, con las contraseñas. No se sube nunca a un repositorio:

```
DB_ROOT_PASSWORD=CambiaEsto_Root_2026
DB_PASSWORD=CambiaEsto_App_2026
```

Fichero `~/tienda/compose.yaml`:

```yaml
services:
  web:
    image: nginx:stable
    ports:
      - "8090:80"
    volumes:
      - ./proxy.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app

  app:
    build: .
    image: tienda:1.0
```

```
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://db:3306/tienda
      SPRING_DATASOURCE_USERNAME: tienda
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy

  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MARIADB_DATABASE: tienda
      MARIADB_USER: tienda
      MARIADB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - datos-db:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 5s
      retries: 10

volumes:
  datos-db:
```

Detalles de la configuración:

- Solo el servicio `web` publica un puerto. La aplicación y la base de datos no son accesibles desde fuera: solo desde la red interna de Compose.
- `depends_on` con `service_healthy` hace que la aplicación espere a que la base de datos esté lista.
- Las variables `SPRING_DATASOURCE_*` muestran cómo se pasa a Spring Boot la conexión a la base de datos desde fuera del código. La aplicación de ejemplo todavía no usa la base de datos; las usaría si tuviera la dependencia de acceso a datos.

```
cd ~/tienda
docker compose config
docker compose up -d --build
docker compose ps
```

Comprobación:

```
curl http://localhost:8090/saludo
docker compose exec web getent hosts app
docker compose exec db mariadb -utienda -p"CambiaEsto_App_2026" -e "SHOW DATABASES;"
```

- La primera orden devuelve el saludo a través de Nginx.
- La segunda muestra la dirección interna del servicio `app`, que el DNS interno de Docker resuelve por su nombre.
- La tercera muestra la base de datos `tienda`.

1.4.5 Pruebas de funcionamiento y de rendimiento

Qué se comprueba	Orden
Estado de los servicios	`docker compose ps`
Mensajes de error	`docker compose logs app`
Tiempo de una petición	`curl -o /dev/null -s -w "%{http_code} %{time_total}s\n" http://localhost:8090/saludo`
Rendimiento con carga	`ab -n 2000 -c 50 http://localhost:8090/saludo`
Consumo de CPU y memoria	`docker stats --no-stream`
Persistencia de los datos	`docker compose down` y `docker compose up -d`: la base de datos conserva su contenido

Práctica 1.12. Medir el rendimiento de la aplicación en contenedores.

```
ab -n 2000 -c 50 http://localhost:8090/saludo
docker stats --no-stream
```

Se anotan:

- las peticiones por segundo (`Requests per second`);
- el tiempo medio por petición (`Time per request`);
- las peticiones fallidas (`Failed requests`, que deben ser 0);
- la memoria que usa cada contenedor.

Después se repite la prueba directamente contra la aplicación, sin pasar por Nginx:

```
docker network ls
docker run --rm --network tienda_default httpd:2.4 ab -n 2000 -c 50
 ↪  http://app:8080/saludo
```

La segunda orden ejecuta `ab` desde un contenedor temporal de la imagen oficial `httpd`, conectado a la red de la aplicación. El nombre de la red (`tienda_default`) es el que muestra `docker network ls`. La diferencia entre las dos mediciones es el coste de la capa de presentación (el proxy).

Al terminar:

```
docker compose down
```

Documentación del despliegue con contenedores (RA1.i)

La documentación debe incluir:

1. el Dockerfile y el `compose.yaml`, comentados;
2. un esquema de los servicios, sus puertos y la red;
3. las imágenes usadas, con su versión (etiqueta);
4. cómo se pasan las contraseñas (fichero `.env`, nunca en el código ni en la imagen);
5. las pruebas de funcionamiento y rendimiento, con sus resultados.

Cierre de la unidad

Resumen

Apartado	Ideas principales
Despliegue	Desplegar es instalar y configurar una aplicación en un servidor para que se pueda usar. Hay que conocer su arquitectura, sus recursos y sus requisitos. Apache sirve contenido y hace de proxy; Tomcat ejecuta aplicaciones Java; Spring Boot lleva su propio Tomcat embebido
DNS	Traduce nombres en direcciones IP mediante un sistema jerárquico y distribuido. BIND 9 permite crear zonas propias. Al desplegar se añaden registros (A, CNAME, MX, TXT, CAA) y se controla el TTL
LDAP	Un directorio guarda los usuarios y grupos de la organización en un árbol. Las aplicaciones validan a los usuarios contra él (búsqueda con una cuenta de servicio y bind con el DN del usuario), así que cada persona tiene una sola cuenta para todo
Contenedores	Una imagen de Docker contiene la aplicación y todo lo que necesita. Un Dockerfile describe cómo construirla y Docker Compose despliega varios contenedores que forman una aplicación

Prácticas de la unidad

Práctica	Contenido	Criterios
1.1	Instalar Apache y Tomcat 11	RA1.c, RA1.e, RA1.f
1.2	Aplicación Spring Boot: ejecutable y desplegada en Tomcat	RA1.e, RA1.g
1.3	Servidor DNS con BIND 9: zonas directa e inversa	RA5.b
1.4	Registros DNS para el despliegue	RA5.b
1.5	Comprobación del DNS con dig, nslookup y resolvectl	RA5.a, RA5.b, RA5.g

Práctica	Contenido	Criterios
1.6	OpenLDAP: instalación, estructura, usuarios y grupos	RA5.c, RA5.d
1.7	Cuenta de servicio, reglas de acceso y validación de usuarios	RA5.e
1.8	Aplicación Spring Boot que valida a sus usuarios contra el directorio	RA5.f
1.9	Docker: imágenes, contenedores y volúmenes	RA1.c
1.10	Dockerfile de una aplicación Spring Boot	RA1.e, RA1.g
1.11	Docker Compose: tres capas	RA1.a, RA1.e
1.12	Pruebas de rendimiento	RA1.f, RA1.i

Actividades de repaso

Despliegue y arquitecturas

1. Explica la diferencia entre los entornos de desarrollo, pruebas y producción.
2. Diferencia entre la arquitectura en tres capas y el patrón MVC.
3. Indica dos ventajas y dos inconvenientes de los microservicios frente a un monolito.
4. Diferencia entre un servidor web y un servidor de aplicaciones. Pon un ejemplo de cada uno.
5. ¿Qué contiene la carpeta `WEB-INF` de un WAR y por qué no es accesible desde el navegador?
6. ¿En qué dirección se publica una aplicación si se copia `ventas.war` en la carpeta `webapps` de Tomcat? ¿Y si se copia `ROOT.war`?
7. Describe las dos formas de desplegar una aplicación Spring Boot.
8. ¿Por qué una aplicación con paquetes `javax.*` no funciona en Tomcat 11?
9. Enumera seis requisitos del proceso de implantación de una aplicación web.

DNS

10. Diferencia entre un sistema de nombres plano y uno jerárquico.
11. Diferencia entre dominio y zona.
12. Diferencia entre una consulta recursiva y una iterativa.

13. ¿Qué indica el indicador `aa` en la respuesta de `dig`?
14. Escribe el nombre que se consulta en la resolución inversa de `10.0.5.20`.
15. ¿Qué ocurre si se modifica una zona sin aumentar el serial del registro SOA?
16. ¿Qué hay que hacer con el TTL antes de cambiar la dirección IP de un servidor?
17. ¿Por qué no puede ponerse un CNAME en el nombre de la propia zona?

LDAP

18. Indica dos diferencias entre un directorio LDAP y una base de datos relacional.
19. Escribe el DN de la usuaria `marta` dentro de `ou=alumnos` en el dominio `ies.test`.
20. ¿Qué define una clase de objeto? Diferencia entre `posixGroup` y `groupOfNames`.
21. ¿Por qué la configuración de OpenLDAP no se edita a mano en `/etc/ldap/slapd.d`?
22. Describe los pasos con los que una aplicación valida a un usuario contra el directorio.
23. ¿Qué significa `by anonymous auth` en una regla de acceso? ¿Qué riesgo tiene el bind simple sin TLS?
24. Enumera los parámetros que necesita una aplicación para validar a sus usuarios contra LDAP.

Contenedores

25. Indica tres diferencias entre una máquina virtual y un contenedor.
26. ¿En qué modelo de nube instala el cliente el sistema operativo?
27. Diferencia entre imagen y contenedor.
28. ¿Qué ventaja tiene una construcción en dos etapas en un Dockerfile?
29. ¿Cómo localiza un contenedor de Compose a otro servicio de la misma aplicación?
30. ¿Por qué el servicio de base de datos no publica ningún puerto en el `compose.yaml` de la práctica 1.11?

Referencias

Libro del módulo

- McGraw Hill, *Despliegue de aplicaciones web*, 1.ª edición, 2022 (ISBN 978-84-486-3274-8).

Normas (RFC)

- RFC 1034 y RFC 1035: el sistema DNS. https://www.rfc-editor.org/rfc/rfc1035
- RFC 6761: nombres de dominio de uso especial (`.test`). https://www.rfc-editor.org/rfc/rfc6761
- RFC 8659: registro CAA. https://www.rfc-editor.org/rfc/rfc8659
- RFC 4511: el protocolo LDAP. https://www.rfc-editor.org/rfc/rfc4511
- RFC 4512: modelo de información del directorio. https://www.rfc-editor.org/rfc/rfc4512
- RFC 4513: métodos de autenticación en LDAP. https://www.rfc-editor.org/rfc/rfc4513

- RFC 4515: filtros de búsqueda LDAP. https://www.rfc-editor.org/rfc/rfc4515
- RFC 2849: formato LDIF. https://www.rfc-editor.org/rfc/rfc2849

Documentación técnica

- Ubuntu Server: https://documentation.ubuntu.com/server/
- Apache HTTP Server 2.4: https://httpd.apache.org/docs/2.4/es/
- Apache Tomcat 11: https://tomcat.apache.org/tomcat-11.0-doc/
- Spring Boot: https://docs.spring.io/spring-boot/
- Spring Security, autenticación LDAP: https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/ldap.html
- BIND 9: https://bind9.readthedocs.io/
- OpenLDAP, guía del administrador: https://www.openldap.org/doc/admin26/
- Apache, módulo `mod_authnz_ldap`: https://httpd.apache.org/docs/2.4/mod/mod_authnz_ldap.html
- Docker: https://docs.docker.com/
- NIST SP 800-145, definición de computación en la nube: https://csrc.nist.gov/pubs/sp/800/145/final